

SERVICIO NACIONAL DE APRENDIZAJE SENA

Tecnología en Analisis y Desarrollo de Software ADSO

MINI-PROYECTO

Normalizacion de un Sistema de Ventas

Modulo 2 - Bases de Datos Relacionales con PostgreSQL

Aprendiz: Johan Daniel Millan Instructor: Jeisson Contreras Centro de Gestion de Mercados, Logistica y Tecnologias de la Informacion

Bogota D.C., 2026

1. Diagnostico de la tabla cruda

La tabla **ventas_crudas** presenta varios problemas de diseno que impiden un manejo correcto de los datos.

Atributos no atomicos

Estos campos tienen varios valores separados por comas en una sola celda, lo que incumple la Primera Forma Normal:

Campo	Ejemplo
productos_codigos	'P001,P002'
productos_nombres	'Mouse USB,Teclado mecanico'
categorias	'Perifericos,Perifericos'
cantidades	'2,1'
precios_unitarios	'45000,180000'
descuentos	'0,0'

Datos repetidos

- Los datos del cliente (nombre, email, telefono, direccion, ciudad) se repiten en cada venta que realiza ese mismo cliente.
- Los datos del vendedor (nombre, zona) se repiten en cada venta que atiende.
- La categoria se repite por cada producto que pertenece a ella.

Anomalias

Insercion: No se puede registrar un producto nuevo sin asociarlo a una venta. Si llega un producto nuevo al catalogo antes de venderse, no hay donde guardarlo.

Actualizacion: Si Maria Gomez cambia su telefono, hay que actualizar todas las filas donde aparece CC101. Si se olvida una, quedan datos contradictorios en la misma base.

Eliminacion: Si se borra la venta V1002 (unica venta de Juan Perez), se pierde toda la informacion del cliente, aunque se quisiera conservar su registro.

Dependencias funcionales

Dependencia	Tipo
cliente_doc → nombre, email, telefono, direccion, ciudad	Total
vendedor_id → nombre, zona	Total
producto_codigo → nombre, categoria	Total
categoria → nombre_categoria	Transitiva
venta_id → fecha, cliente_doc, vendedor_id, metodo_pago	Total
venta_id + producto_codigo → cantidad, precio_unitario, descuento	Compuesta

2. Primera Forma Normal (1FN)

Para cumplir la 1FN hay que eliminar los grupos repetitivos. Cada fila que tenia varios productos separados por comas se convierte en varias filas, una por producto.

Antes:

```
V1001 | CC101 | 'P001,P002' | '2,1' | '45000,180000'
```

Despues:

```
V1001 | CC101 | P001 | 2 | 45000  
V1001 | CC101 | P002 | 1 | 180000
```

La clave compuesta provisional es (**venta_id**, **producto_codigo**) porque identifica de forma unica cada linea de detalle.

Problema que queda pendiente: los datos de cliente y vendedor siguen repetidos en cada fila porque todavia dependen solo de una parte de la clave.

3. Segunda Forma Normal (2FN)

Para cumplir la 2FN hay que eliminar las dependencias parciales. Un atributo que depende solo de una parte de la clave compuesta debe salir a su propia tabla.

Atributo	Depende de	Tabla resultante
nombre, email, telefono, direccion, ciudad del cliente	solo cliente_doc	clientes
nombre, zona del vendedor	solo vendedor_id	vendedores
nombre, categoria del producto	solo producto_codigo	productos
fecha, metodo_pago	solo venta_id	ventas
cantidad, precio_unitario, descuento	venta_id + producto_codigo	detalle_venta

4. Tercera Forma Normal (3FN)

Para cumplir la 3FN hay que eliminar las dependencias transitivas. Un atributo no debe depender de otro atributo que no sea la clave.

Categoria: depende del producto, no de la venta. Si se cambia el nombre de una categoria hay que actualizar todos los productos. Se crea la tabla **categorias** para manejarla de forma independiente.

Metodo de pago: **entidad_pago** depende de **metodo_pago**, no directamente de la venta. Se crea la tabla **metodos_pago** para evitar que el mismo metodo se escriba diferente en distintas ventas.

total_venta: no se almacena porque es un dato calculable desde el detalle con la formula **SUM(cantidad * precio_unitario - descuento)**. Guardarlo crearia riesgo de inconsistencias.

precio_unitario en detalle_venta: se guarda en el detalle y no se toma del catalogo. Esto es porque el precio de un producto puede cambiar en el futuro, pero el precio de una venta ya realizada no debe cambiar.

5. Modelo final

clientes

	Campo	Tipo
PK	cliente_doc	VARCHAR(20)
	cliente_nombre	VARCHAR(100) NOT NULL
	cliente_email	VARCHAR(120) UNIQUE
	cliente_telefono	VARCHAR(30)
	cliente_direccion	VARCHAR(150)
	cliente_ciudad	VARCHAR(80)

vendedores

	Campo	Tipo
PK	vendedor_id	VARCHAR(10)

	Campo	Tipo
	vendedor_nombre	VARCHAR(100) NOT NULL
	vendedor_zona	VARCHAR(80)

categorias

	Campo	Tipo
PK	categoria_id	SERIAL
	categoria_nombre	VARCHAR(80) UNIQUE NOT NULL

productos

	Campo	Tipo
PK	producto_codigo	VARCHAR(10)
	producto_nombre	VARCHAR(100) NOT NULL
	precio_catalogo	NUMERIC(12,2) CHECK >= 0
FK	categoria_id	INT → categorias

metodos_pago

	Campo	Tipo
PK	metodo_id	SERIAL
	metodo_nombre	VARCHAR(80) UNIQUE NOT NULL
	entidad_pago	VARCHAR(80)

ventas

	Campo	Tipo
PK	venta_id	VARCHAR(10)
	fecha_venta	DATE NOT NULL
FK	cliente_doc	VARCHAR(20) → clientes
FK	vendedor_id	VARCHAR(10) → vendedores
FK	metodo_id	INT → metodos_pago

detalle_venta

	Campo	Tipo
PK FK	venta_id	VARCHAR(10) → ventas

	Campo	Tipo
PK FK	producto_codigo	VARCHAR(10) → productos
	cantidad	INT CHECK > 0
	precio_unitario	NUMERIC(12,2) CHECK >= 0
	descuento	NUMERIC(12,2) DEFAULT 0 CHECK >= 0

6. Checklist final

- La tabla final no contiene campos con valores separados por comas.
- Cada tabla tiene una clave primaria.
- Las relaciones tienen claves foraneas activas.
- Las cantidades no aceptan valores menores o iguales a cero.
- Los productos no se duplican.
- Los clientes no se duplican por cada compra.
- El total de la venta se calcula con SQL desde detalle_venta.
- El diagrama ER coincide con los scripts SQL.
- El README explica como ejecutar los scripts.
- Las consultas de validacion se ejecutan sin errores.